NES ROM Translation

Full Step-by-Step Guide and Mighty Final Fight Case Study

ROM Translation Workflow Documentation

Step 20 reference release

Contents

Case study and release scope

What the final project contains

Source identity and ROM preflight

Mapper, CPU and PPU layout

Text extraction and indexing

Render pipeline analysis

Vietnamese font design

Translation and fixed-slot encoding

Safe insertion and ROM expansion

IPS and reproducible builds

FCEUX/Mesen test plan

Troubleshooting guide

Project history, Steps 1-20

Step 20 file map

External references

Release definition

Case study: Mighty Final Fight (USA) to Vietnamese

Document purpose. This guide records the workflow, decisions, failure modes, tools, checks, and release boundaries demonstrated by the Mighty Final Fight Vietnamese translation project through Step 20. It is designed to be repeatable for other NES/Famicom games, but addresses, mappings, banks, pointer structures, and control codes in this case are game-specific and must not be copied blindly to another ROM.

1. What the final project contains

The Step 20 build translates the current verified pointer-indexed text inventory:

7 opening/cutscene records.

78 boss-dialogue index rows.

5 credits records.

90 index-level catalog rows total.

Four boss-index aliases share already existing pointers, so the 90 rows correspond to 86 distinct physical slots.

The output also incorporates the expanded CHR font/bank fix used for the opening and the localized dialogue pilot. The opening and first boss dialogue were reported by the tester to render cleanly in earlier steps. The full Step 20 build was **not** yet confirmed by a complete FCEUX playthrough. Menu/HUD graphics-text and any unknown text path outside the verified pointer inventory are not claimed translated.

Release files

Mighty_Final_Fight_Vietnamese_FULL_TRANSLATION_STEP20.nes - cumulative output ROM, 278,544 bytes.

Mighty_Final_Fight_Vietnamese_FULL_TRANSLATION_STEP20.ips - cumulative IPS patch intended for the clean USA source, 20,208 bytes and 137 records.

step20_full_translation_catalog.tsv - per-index English/Vietnamese catalog.

pointer_tables.csv - discovered text pointers and physical offsets.

mighty_final_fight.tbl - verified base byte-to-character table and special/control byte annotations.

Build, validator, scope, and checksum files.

Do not apply the cumulative IPS on top of an already patched ROM. Do not redistribute the user's clean source ROM as part of a public project pack.

2. Reproducible source identity and ROM preflight

Always begin with a clean, unmodified ROM. Keep it read-only and write all outputs to separate paths. For this project, the expected source SHA-256 is:

```
5e70be9de1c0d272c456873216a568eb2377f5c211456c8f8330c601f3920b26
```

Original ROM facts recorded during the project:

Filename: Mighty Final Fight (USA).nes

Size: 262,160 bytes (16-byte iNES header plus 128 KiB PRG and 128 KiB CHR data).

NES 2.0 format; mapper 4 (MMC3); horizontal mirroring reported in the initial analysis.

Original body CRC32: 3F78037C.

Original PRG CRC32: 2EE3EF15.

Original CHR CRC32: BCFEB409.

Original body MD5: be4670792b17c74eecf3a0e7f37ecfe2.

These values identify this exact case-study input. On a new project, derive the values from the user's clean source; do not paste these expected hashes into another project's configuration.

Recommended intake commands:

```
sha256sum "Mighty Final Fight (USA).nes"
file "Mighty Final Fight (USA).nes"
xxd -l 16 "Mighty Final Fight (USA).nes"
```

Create an intake manifest before editing: source filename, source hash, byte length, header format, mapper/submapper if present, PRG/CHR sizes, trainer flag, mirroring, tool versions, output locations, and legal/source provenance notes. A hash mismatch is a stop condition, not a warning to ignore.

3. Determine the mapper and CPU/PPU layout

The case-study ROM uses MMC3 (mapper 4). MMC3 changes PRG and CHR banks in response to writes to mapper registers, so a CPU address alone is not enough to identify bytes in the ROM. Record both:

CPU address and active CPU PRG window/bank.

Physical ROM offset and the formula that maps the active bank to that offset.

Do not assume that the same CPU address points to the same physical bytes at every point in gameplay. A bank can be switchable in one context and fixed in another. Similarly, a visible PPU tile ID can fetch different graphics depending on which CHR bank is active.

For NES ROMs, the header is 16 bytes. For the original project, the PRG data starts at file offset 0x10; CHR data starts at 0x10 + 0x20000. Always document whether an offset is a CPU address, PRG-relative offset, or absolute file offset. Label them explicitly to prevent off-by-16 mistakes.

4. Extract and index the text

4.1 Find likely text, then prove the decoder

Start with a byte scan for readable runs, but do not trust apparent text until it is cross-checked against an in-game screen. An extraction is only a hypothesis until the byte mapping, control bytes, pointer boundary, and rendered result agree.

The verified table for this game begins:

```
00 = space
10-29 = A-Z
2A = apostrophe
```

2B = period
2C = comma

The .tbl file contains extra annotations for special/control bytes, including 75, 76, 78, F9, FA, FB, FC, FE, and FF. These are not all printable characters. Byte 0x35 was specifically found in boss-dialogue records as metadata/header material and must not be assigned casually as a glyph.

Do not map every byte by visual guess. Some values are tile indices, metadata, timing markers, animation commands, line breaks, or terminators. Keep the text table and control-code notes separate.

4.2 Pointer tables and record boundaries

The Step 20 inventory includes three groups:

Opening: 7 records, pointer table at PRG offsets 0x1B2C/0x1B33; pointer targets include CPU $9B3A, $9C34, $9C8C, $9CA3, $9D2E, $9D5C, and $9E07. The opening text data is in PRG bank 0 mapped at $8000 during the known path.

Boss dialogue: 78 index entries using high-pointer table $A4FD and low-pointer table $A54C; records are found in the region around PRG offset 0x09D71 through 0x0A4CA. The boss records carry a five-byte metadata prefix before the visible payload in the verified extraction model.

Credits: 5 records, low-pointer table $A7D6, high-pointer table $A7DB, pointer targets $A7E0, $A7F1, $A87D, $A8CB, and $A95C (addresses are CPU/bank context from the case analysis; always consult pointer_tables.csv for PRG and file offsets).

Use the provided pointer_tables.csv rather than retyping these values. Maintain two identifiers for every record: a logical index (the item the game uses) and a physical slot (the bytes that are actually replaced). If two logical indices point to the same physical bytes, keep both index rows in the catalog and mark their shared pointer. Do not duplicate writes to the same slot without a deliberate policy.

4.3 Build a catalog before translating

Every record should carry:

group and logical ID;

CPU pointer and pointer-table address;

PRG-relative and absolute file offsets;

original capacity / terminator location;

English source text;

decoded control-code representation;

Vietnamese draft;

encoded bytes and byte count;

shared pointer / alias information;

status, notes, and emulator test state.

The catalog is the source of truth. Keep extraction, translation, encoding, and injection separate so the translation can be corrected without repeating reverse engineering.

5. Understand the render pipeline before modifying text

A ROM can have different renderers for opening narration, boss dialogue, credits, menus, and HUD. Do not assume that one hook handles them all.

The project located a common character/PPU path and traced these important routines/contexts:

Opening text hook candidate at CPU $9A47 in physical PRG bank 0 for the observed context.

Text/VRAM queue RAM around $0780-$0784.

VRAM queue consumer at CPU $C1FD in fixed PRG bank 14 for observed events.

Character lookup routine at CPU $F406 in fixed PRG bank 15 for observed events.

Those addresses are case-specific and only describe the observed trace. The FCEUX trace ran to its 100,001-event limit, so it did not constitute a full-game trace. The active PRG/CHR mapping must be recorded whenever a candidate hook or tile is evaluated.

Required observations

Which routine reads one text byte?

Which routine writes it into RAM or the PPU queue?

Which PPU address is updated ($2006/$2007 path)?

Which CHR bank supplies the resulting tile at that moment?

What values represent newline, end-of-string, clear, animation, and metadata?

Does the same routine run in opening, boss, credits, menu, and HUD contexts?

Use a debugger breakpoint, code/data logger, trace logger, or emulator Lua callback where supported. Do not assume every FCEUX build offers the same Lua API. In this project, memory.registerread was absent and caused an early Lua failure; the corrected script checks capabilities before registering the callback and marks unavailable observations rather than guessing values.

6. Font design for Vietnamese

6.1 The first composite-glyph approach

The early builds drew a complete accented Vietnamese letter into one 8x8 tile. This made it easy to render when the original renderer writes one character byte to one tile, but some letters became cramped. User testing found that Ị looked too short, Đ resembled B, and composite tone marks made TỒN look like TỐN.

The early prototype also had real byte/bank problems: high-byte glyph codes 0x80-0x8F looked like unrelated graphics because the active MMC3 CHR bank did not contain the custom glyphs. Redrawing a glyph cannot repair a wrong bank mapping.

6.2 Bank-safe composite glyphs used for the confirmed opening

FCEUX traces showed that the opening path could run with CHR register R0 set to 0x78 while R1 varied (0x62 on many hits and 0x56 on others). The custom high-byte tiles had initially been placed in a bank that was not the active bank for the relevant tile half. Step 17 expanded CHR ROM by 16 KiB, copied the

observed source-bank contexts, and placed matching custom glyph slots into the copied bank contexts. The user confirmed the Step 17 glyphs rendered correctly and there was no transition issue.

Step 20 deliberately avoids new 0x80-0x8F characters in additional boss/credits payloads because high-byte routing was only explicitly confirmed in the opening path. It retains the working opening mapping and uses known low-byte glyphs for the added boss/credits text.

6.3 Split-letter/diacritic method from the Contra/Kontora article

The article by STONE BOAT describes a different but valuable model: draw the base letter in one tile and a tone mark in a separate tile row, then map a logical accented character to the base glyph plus a mark. For letters with both an upper mark and a dot below, use an upper mark, base glyph, and lower-dot placement. The article also explains why code should be inserted via a bank-aware jump to a verified free region rather than growing an existing routine in place and shifting later addresses. Its example is for a different Contra/Kontora ROM and mapper; its addresses and mapper writes are not reusable here.

Reference: https://sites.google.com/view/sfc65816/famicom-nes/contra-v%C3%A0-kontora?authuser=0

A split-mark renderer can reduce the number of complete accented glyph tiles and improve typography, but it is not a drop-in change. It must account for PPU tile-row placement, line spacing, line overlap, punctuation, clearing behavior, and the game's sprite/background timing. The project generated split-accent art as an exploration, but the released Step 20 build uses the known-good composite-bank approach for opening text and stays conservative on extended glyphs in boss/credits records.

6.4 Font quality checklist

Test every family of forms, not just one sample word:

A/E/I/O/U with acute, grave, hook, tilde, and dot below.

Circumflex and breve combined with every supported tone mark.

Đ/đ, Ă/ă, Â/â, Ê/ê, Ô/ô, Ơ/ơ, and Ư/ư.

Distinguish Ồ from Ố, Ủ from Ú, Ở from Ớ, Ờ from Ớ, and Ị from I.

Check uppercase and lowercase if both are used.

Check baseline, caps height, left/right padding, punctuation alignment, and visible contrast at native 8x8 scale.

7. Translation policy and fixed-slot encoding

Translate meaning naturally, but obey the game's byte and layout limits. For each record:

Read the full English context, including previous/next lines and speaker/scene.

Draft Vietnamese with full diacritics.

Fit it to the original slot or screen line limits; rephrase instead of deleting necessary accents.

Encode only through a verified table.

Reject unmapped characters.

Reject reserved metadata/control bytes in normal text.

Check byte count, not Unicode character count.

Preserve terminators, line-break commands, animation markers, and any metadata that the renderer uses.

Review all aliases (logical indices with shared pointers).

For this game, the Step 20 encoder rejects glyph codes >= 0x80 for newly changed boss/credits payloads, and rejects 0x35. This is a deliberate safe subset, not a universal NES rule. The opening uses the special bank-aware high-byte glyph routing that was tested in earlier steps.

Do not overwrite text in place if it exceeds the original slot. Options include concise translation, relocating text and updating pointers, or ROM expansion; each must be justified by the game's actual pointer scheme and mapper. Relocation is not a casual first move because all consumers of the pointer may need updates.

8. Safe insertion and ROM expansion

For any new code or data:

Scan candidate fill bytes (00 or FF) but do not treat fill as proof that a location is unused.

Search direct and indirect references to the region.

Determine which CPU address/window maps the physical bank at runtime.

Check bank state at entry and during interrupts/NMI.

If code is in another bank, prove a reachable mapping/return path; do not call code in a bank that is not currently mapped.

Preserve original code addresses where possible: replace a short entry sequence with a jump/call to a helper, keeping the helper in a validated location.

Preserve CPU registers/flags and stack semantics exactly where the caller expects them.

Test the non-target path, adjacent address, first/middle/last characters, and state transitions.

If expanding ROM, update the header according to the ROM's actual format and validate that the emulator detects the new PRG/CHR dimensions and mapper.

A Step 12 helper bug illustrates why it matters: the helper overwrote A while checking PPU address and then wrote the address byte as a character when the target condition was false. Step 13 corrected it by preserving/restoring the accumulator and processor status. Automated byte/logic tests were useful, but the real proof was the user's emulator observation that only the intended T changed and other characters stayed legible.

9. IPS and reproducible builds

Build from the clean source each time. A cumulative patch must be defined against the clean source, even when its build depends internally on a prior known-good baseline.

Expected Step 20 source/output values:

Clean source SHA-256: 5e70be9de1c0d272c456873216a568eb2377f5c211456c8f8330c601f3920b26.

Step 20 ROM SHA-256: 03d2d05e2303e2b41a5213b248bdd3d20cfa478a8e473dd00734d42882e6bcbc.

Step 20 IPS SHA-256: 2778df5a7eabad3ca66d32b2a9fc6e41f1b0b7a825daa8c174e43dbdc6b135b5.

ROM length: 278,544 bytes.

IPS length: 20,208 bytes, 137 records.

Rebuild command from the Step 20 project directory (substitute real paths):

```
python3 tools/build_full_translation_step20.py \
  --source-rom "/path/to/Mighty Final Fight (USA).nes" \
  --project "/path/to/Mighty_Final_Fight_Vietnamese_Project_step20" \
  --base-ips "/path/to/Mighty_Final_Fight_Vietnamese_Project_step20/analysis/step19_boss_dialogue_pilot_combined.ips" \
  --output-rom "/path/to/Mighty_Final_Fight_Vietnamese_FULL_TRANSLATION_STEP20.nes" \
  --output-ips "/path/to/Mighty_Final_Fight_Vietnamese_FULL_TRANSLATION_STEP20.ips"
```

Run independent validation from the project directory:

```
python3 tools/validate_final_build_step20.py \
  --source-rom "/path/to/Mighty Final Fight (USA).nes"
```

The validator checks source identity, expected output hashes, IPS parsing, exact clean-source round-trip, catalog row count/group counts, empty translation fields, preservation of the Step 19 baseline, and grunt animation bytes. Use these as project-specific examples, not constants for another ROM.

10. Test plan in FCEUX/Mesen

Use the latest output built from the clean source. Save screenshots and log the exact ROM hash tested.

Smoke test

ROM opens and powers on.

Title screen works.

Opening page 1 through 7 displays without malformed glyphs.

Text clears/advances at the right time.

Transition out of opening is clean.

Dialogue test

Reach each boss dialogue class represented in the catalog.

Check every used glyph family, especially high-risk Vietnamese forms.

Confirm shared-pointer aliases show consistent text.

Confirm localized grunts preserve punctuation and animation timing.

Check line lengths and text-box boundaries.

Credits test

Reach credits, check all 5 indexed records, line breaks, end marker, and return/reset behavior.

Regression test

Gameplay, sprites, background animations, music, pause, damage/death/retry, and scene transitions still work.

Check any CHR bank expansion against unrelated graphics and mapper behavior.

Run for longer than the initial test; a short opening smoke test does not count as full-game verification.

Required statuses

Mark each check as PASS, FAIL, or NOT TESTED. Never label a build emulator-verified because a preview image looks correct or a static validator passed.

11. What to do when a glyph or screen fails

Symptom: an accented character becomes unrelated graphics

Likely cause: wrong active CHR bank or code-to-tile mapping.

First diagnostic: trace MMC3 CHR registers and inspect the PPU tile ID in the failing scene.

Symptom: Ồ looks like Ố

Likely cause: composite-mark artwork is too cramped or ambiguous.

First diagnostic: inspect the 8x8 tile at native scale; adjust mark spacing or adopt split marks only after designing and validating the renderer.

Symptom: a letter changes at unrelated positions

Likely cause: a hook corrupts CPU state or tests only a reusable PPU address.

First diagnostic: check A/X/Y, flags, stack preservation, gate conditions, and non-target cases.

Symptom: the game crashes after code injection

Likely cause: wrong bank mapped, bad return path, branch/relative-offset error, clobbered register, or overwrite of referenced data.

First diagnostic: use breakpoints and verify CPU-address-to-physical-bank mapping at runtime.

Symptom: text is truncated or a page glitches

Likely cause: slot overflow, moved terminator, overwritten control byte, or encoding mismatch.

First diagnostic: compare before/after bytes and validate slot lengths and terminators.

Symptom: dialogue works but credits do not

Likely cause: a different renderer, pointer table, or control-code grammar.

First diagnostic: trace and catalog each text context separately.

Symptom: a Lua script fails because an API is missing

Likely cause: FCEUX version/API mismatch.

First diagnostic: feature-detect callbacks; do not register unsupported functions; mark missing observables as unavailable.

12. Project history: how the workflow evolved (Steps 1-20)

This history is included because the failures taught more than the final static patch:

Steps 1-2: Intake and extraction. Identified the USA ROM, decoded base glyph bytes, found opening/boss/credits pointers, built the text table and inventory.

Step 3: CHR inventory. Decoded all 8x8 2bpp tiles; generated per-bank sheets and candidate glyph plans.

Step 4: Render-path analysis. Found the direct PPU tile path and began matching text bytes to global CHR tile IDs.

Step 5: Font prototype. Made a first full-tone Vietnamese bitmap set and added a reserved-byte audit after discovering 0x35 conflicts with boss record metadata.

Step 6: One-character smoke test. Created an IPS testing a single accented glyph in the opening text.

Steps 7-8: Visual iterations. Used user screenshots to identify ambiguous tone marks and letter shapes; regenerated the font, then tested BẢN DỊCH VIỆT.

Step 9: Split-accent exploration. Prototyped separate tone-mark tiles based on the Contra/Kontora article; documented PPU row/collision risks rather than injecting unproven logic.

Step 10: Static MMC3 fill/bank audit. Scanned PRG banks and candidate fill regions; concluded a fill-byte scan alone cannot prove a safe code cave.

Steps 11-12: Runtime trace and first helper. Built an FCEUX Lua tracer; fixed a callback compatibility issue; traced bank state and built a tiny T -> E helper probe.

Step 13: Regression fix. User screenshots showed widespread corruption; identified that the helper wrote a PPU-address byte instead of preserving A on the non-target path. Corrected register/flag preservation and got user confirmation that only the target T changed.

Step 14: Opening translation pilot. Added a full-tone opening record within its fixed-size slot.

Steps 15-16: Glyph redesign. Corrected compressed letter forms and ambiguous mark placement from user feedback; static checks passed but the emulator still exposed bank-related glitches.

Step 17: Correct CHR bank routing. Trace results showed the custom high-byte glyphs weren't in the active CHR context. Expanded CHR by 16 KiB, duplicated observed source-bank contexts, installed matching glyph slots, and added routing. User confirmed correct rendering and no transition problem.

Step 18: Opening sequence. Translated the remaining opening records; user reported correct clean rendering.

Step 19: Boss pilot. Translated the first Thrasher dialogue batch; user reported it looked fine.

Step 20: Current indexed catalog. Filled the remaining indexed boss and credits records, handled alias pointers and grunt animation bytes, and built the cumulative ROM/IPS. Static validation passed; full-game emulator playthrough remains pending.

The first project request was not finished merely when text bytes had been inserted. The release quality came from repeated loops of source hash checks, dumps, font previews, runtime traces, emulator screenshots, bug diagnosis, rebuilds, and regression tests.

13. File map for the Step 20 case study

The companion project_reference/ directory retains the relevant Step 20 artifacts:

STEP20_README.md

STEP20_TRANSLATION_PLAN.md

mighty_final_fight.tbl

pointer_tables.csv

step20_full_translation_catalog.tsv

english_text_dump.md

step20_scope_report.md

step20_validation.txt

step20_checksums.txt

build_full_translation_step20.py

validate_final_build_step20.py

These are supplied as project reference files. They do not include the clean original ROM.

14. External technical reference

STONE BOAT, “Contra và Kontora,” explains a NES/Famicom text renderer, a logical character mapping that composes a base glyph with separately stored upper/lower diacritic marks, and the bank-aware practice of jumping to a new routine in validated free space instead of shifting existing addresses. The example is for a different title and mapper; adopt the concept, not its addresses, register writes, or code.

Article: https://sites.google.com/view/sfc65816/famicom-nes/contra-v%C3%A0-kontora?authuser=0

FCEUX Lua API / version notes: https://fceux.com/web/help/LuaFunctionsList.html and https://fceux.com/web/help/WhatsNew266.html

NESdev wiki (hardware and mapper background): https://www.nesdev.org/wiki/NES_2.0 and https://www.nesdev.org/wiki/MMC3

15. Release definition

A deliverable should be called **a reproducible indexed-text release** when the clean-source hash is checked, all cataloged text records are mapped/translated, slot/control/pointer protections pass, the IPS round-trip reproduces the exact ROM, and the artifacts are documented.

Call it **emulator-verified** only after the actual output hash has been loaded in an emulator and the opening, relevant dialogue, credits, transitions, and a reasonable gameplay regression pass have been recorded. The Step 20 build met the static release conditions; its full playthrough was still marked pending.